

FICHE TECHNIQUE — CONTRAT D'API REST

Application de gestion des transactions Mobile Money Version : 1.0 (finalisée) — Septembre 2026 Base URL : `/api/v1` — Format : JSON — Authentification : Laravel Sanctum (Bearer Token) — Base de données : PostgreSQL

1. Principes généraux

- Toutes les routes métier nécessitent une authentification (hors `register`, `login`, `forgot-password`, `reset-password`).
- Toutes les données métier sont limitées à l'agent authentifié via `user_id`.
- **Identifiants : UUID** sur toutes les tables (choix assumé pour préparer une évolution offline-first, où le mobile doit pouvoir générer un identifiant avant même de contacter le serveur).
- Les colonnes `version`, `sync_status` et `synced_at` existent sur `transactions` mais **ne sont pas exploitées en V1** — elles préparent la synchronisation offline future sans qu'aucun mécanisme ne les utilise encore.
- **Aucune transaction, aucun client, aucun réseau n'est jamais supprimé physiquement.** Les transactions passent par un `statut` (ANNULEE), les réseaux par un `soft delete` (`deleted_at`).
- L'agent ne manipule jamais directement un `client_id` : un client est identifié par son `telephone`, retrouvé ou créé automatiquement à l'enregistrement d'une transaction.
- Toute création, modification ou annulation de transaction génère automatiquement une entrée dans `transaction_histories`.
- Le téléphone sert d'identifiant de connexion de l'agent ; l'email ne sert qu'à la récupération de mot de passe.

2. Convention des réponses

Succès

```
{ "data": {}, "message": "Opération effectuée avec succès." }
```

Pour une liste :

```
{ "data": [], "meta": { "current_page": 1, "per_page": 20, "total": 100 } }
```

Erreur de validation — HTTP 422

```
{ "message": "Les données fournies sont invalides.", "errors": { "montant": ["Le montant doit être supérieur à 0."] } }
```

Non authentifié — HTTP 401

```
{ "message": "Unauthenticated." }
```

Ressource inexistante — HTTP 404

```
{ "message": "Ressource introuvable." }
```

3. AUTHENTIFICATION

3.1 POST `/auth/register`

```
{  "name": "Jean KABORE",  "telephone": "70000000",  "email": "agent@example.com",  "password": "*****",  "password_confirmation": "*****"}
```

Validation : `telephone` requis + unique, `email` requis + unique + email valide, `password` requis + confirmé (min. 8 caractères). `code_agent` généré automatiquement côté serveur (ex. AG-88240), jamais saisi par l'agent.

Réponse 201

```
{
  "message": "Compte créé avec succès.",
  "data": {
    "token": "1|xxxxxxxxxxxxxxxx",
    "user": {
      "id": "uuid",
      "name": "Jean KABORE",
      "telephone": "70000000",
      "email": "agent@example.com",
      "code_agent": "AG-88240",
      "photo": null,
      "localisation_point": null
    }
  }
}
```

3.2 POST /auth/login

```
{ "telephone": "70000000", "password": "*****" }
```

Recherche de l'agent par `telephone` (pas par email). Réponse 200 de même forme que le register.

3.3 POST /auth/logout

Authentification requise. Invalide le token courant.

```
{ "message": "Déconnexion réussie." }
```

3.4 GET /auth/me

Authentification requise. Retourne l'agent authentifié (même structure que la réponse login).

3.5 PATCH /auth/me

```
{ "name": "Jean KABORE", "email": "nouveau@example.com", "photo": "...", "localisation_point": "Secteur 15, Ouaga" }
```

Modifiable : `name`, `email`, `photo`, `localisation_point`. **Non modifiable :** `telephone` (identifiant de connexion), `code_agent` (identifiant métier fixe).

3.6 POST /auth/change-password

Authentification requise.

```
{ "current_password": "*****", "password": "*****", "password_confirmation": "*****" }
```

Vérifie `current_password`. Révoque tous les tokens Sanctum de l'agent **sauf** celui de la requête courante (déconnexion des autres appareils uniquement).

```
{ "message": "Mot de passe modifié avec succès." }
```

3.7 POST /auth/forgot-password

```
{ "email": "agent@example.com" }
```

Génère un code à 6 chiffres, valable 15 minutes, envoyé par email (pas de lien cliquable — évite d'avoir à héberger une page web pour une app React Native CLI). Réponse générique systématique, que l'email existe ou non :

```
{ "message": "Si un compte existe avec cet email, un code a été envoyé." }
```

Route à protéger par un throttle (ex. 3 tentatives/minute) pour éviter le spam d'envoi.

3.8 POST /auth/reset-password

```
{ "email": "agent@example.com", "code": "482910", "password":
"*****", "password_confirmation": "*****" }
```

Vérifie le code et son expiration (15 min), met à jour le mot de passe, **révoque tous les tokens Sanctum** de l'agent (reconnexion obligatoire partout, y compris l'appareil ayant fait la demande).

4. TABLEAU DE BORD

4.1 GET /dashboard

Authentification requise. Toujours calculé sur la journée courante — aucun paramètre de date (pour consulter un autre jour, on utilise GET /transactions avec date_debut / date_fin).

Réponse 200

```
{
  "data": {
    "date": "2026-09-12",
    "nombre_transactions": 12,
    "total_depots": "450000.00",
    "total_retraits": "320000.00",
    "dernieres_transactions": [
      {
        "id": "uuid",
        "type_operation": "depot",
        "montant": "15000.00",
        "reference": "0M-260912-0034",
        "statut": "ENREGISTREE",
        "created_at": "2026-09-12T14:20:00Z",
        "client": { "nom": "Sawadogo", "prenoms": "Alizeta",
"telephone": "70000000" },
        "reseau": { "nom": "Orange Money", "code":
"orange_money" }
      },
    ],
    "derniers_soldes": [
      { "reseau": { "id": "uuid", "nom": "Orange Money",
"code": "orange_money" }, "solde": "450000.00" },
      { "reseau": { "id": "uuid", "nom": "Moov Money", "code":
"moov_money" }, "solde": null }
    ]
  }
}
```

Règles : - `dernieres_transactions` : les 5 plus récentes, non paginées.
- `derniers_soldes` : une entrée pour **chaque réseau existant**, même sans transaction associée (`solde: null` dans ce cas — affiché “Aucune donnée” côté mobile, jamais “0 FCFA”, pour ne pas laisser croire à un solde réellement nul). - Aucun calcul de trésorerie n'est effectué : `solde` est simplement la dernière valeur `solde_apres_operation` saisie par l'agent pour ce réseau.

5. RÉSEAUX / SERVICES

5.1 GET /reseaux

Authentification requise.

GET /api/v1/reseaux → réseaux actifs uniquement (non supprimés)

GET /api/v1/reseaux?avec_supprimes=true → tous, y compris supprimés (écran de gestion)

5.2 POST /reseaux

```
{ "nom": "Wave", "code": "wave", "logo": null }
```

code requis, unique, **immuable après création**.

5.3 PATCH /reseaux/{reseau}

```
{ "nom": "Wave Money", "logo": "https://..." }
```

Modifiable : nom, logo. Non modifiable : code.

5.4 DELETE /reseaux/{reseau}

Soft delete (deleted_at) — le réseau disparaît des choix proposés pour une nouvelle transaction, mais reste visible dans l'historique de toutes les transactions déjà enregistrées avec ce réseau (la contrainte `RESTRICT` sur `transactions.reseau_id` n'est ainsi jamais mise en défaut).

```
{ "message": "Réseau supprimé avec succès." }
```

6. CLIENTS

6.1 GET /clients

Authentification requise. Paramètres : search (téléphone, nom, prénoms), telephone, page, per_page.

Réponse 200

```
{
  "data": [
    {
      "id": "uuid",
      "telephone": "70000000",
      "nom": "KABORE",
      "prenoms": "Paul",
      "nombre_transactions": 24,
      "derniere_transaction_at": "2026-09-11T10:12:00Z"
    }
  ],
  "meta": { "current_page": 1, "per_page": 20, "total": 5 }
}
```

6.2 GET /clients/lookup

GET /api/v1/clients/lookup?telephone=70000000

Authentification requise. Un seul paramètre : telephone. Utilisé pendant la saisie du formulaire Nouvelle Transaction pour préremplir le nom si le client existe déjà. Ne crée jamais de client.

```
{ "data": { "found": true, "client": { "id": "uuid", "telephone": "70000000", "nom": "KABORE", "prenoms": "Paul" } } }
```

```
{ "data": { "found": false, "client": null } }
```

6.3 GET /clients/{client}

Authentification requise.

```
{
  "data": {
    "id": "uuid",
    "telephone": "70000000",
    "nom": "KABORE",
    "prenoms": "Paul",
    "date_naissance": "1990-05-10",
    "nationalite": "Burkinabè",
    "type_piece": "CNIB",
    "numero_piece": "B1234567",
    "date_expiration_piece": "2030-05-10",
  }
}
```

```

        "created_at": "2025-11-03T08:00:00Z",
        "statistiques": { "nombre_transactions": 14, "total_depots":
"850000.00", "total_retraits": "630000.00" }
    }
}

```

`created_at` alimente le “Client depuis [date]” affiché sur la fiche client.

6.4 GET /clients/{client}/transactions

Authentification requise. Paramètres : `page`, `per_page`. Transactions les plus récentes en premier, même structure que les éléments de `GET /transactions`.

6.5 PATCH /clients/{client}

```

{
  "nom": "KABORE",
  "prenoms": "Paul",
  "date_naissance": "1990-05-10",
  "nationalite": "Burkinabè",
  "type_piece": "CNIB",
  "numero_piece": "B1234567",
  "date_expiration_piece": "2030-05-10"
}

```

Modifiable : `nom`, `prenoms`, `date_naissance`, `nationalite`, `type_piece`, `numero_piece`, `date_expiration_piece`. **Non modifiable** : `telephone` — c’est la clé qui permet de retrouver le client à chaque transaction ; la laisser figée évite tout doublon ou rattachement erroné.

Aucun POST /clients : un client n’existe dans l’application que s’il a déjà fait au moins une transaction. Il est créé automatiquement par `POST /transactions`, jamais manuellement.

7. TRANSACTIONS

7.1 POST /transactions

L’endpoint central de l’application.

```

{
  "telephone": "70000000",
  "nom": "KABORE",
  "prenoms": "Paul",
  "date_naissance": "1990-05-10",
  "nationalite": "Burkinabè",
  "type_piece": "CNIB",
  "numero_piece": "B1234567",
  "date_expiration_piece": "2030-05-10",
  "reseau_id": "uuid",
  "type_operation": "retrait",
  "montant": "70000",
  "solde_apres_operation": "350000",
  "note": null
}

```

Obligatoires : `telephone`, `reseau_id`, `type_operation`, `montant`.

Facultatifs : tout le reste, y compris `nom` / `prenoms` (un client peut être créé avec le seul téléphone).

Traitement (opération atomique, DB::transaction()) :

1. Identifier l’agent authentifié
2. Rechercher le client par `telephone`
3. Trouvé → réutiliser son id ; sinon → créer le client (`nom`/`prenoms` peuvent rester NULL si non fournis)
4. Fusion non destructive : compléter les champs vides du client, ne jamais écraser une donnée déjà renseignée
5. Générer la référence : `{CODE_RESEAU}-{AAMJJ}-{0001}`, `incrément journalier par réseau` — ex. OM-260912-0034
6. Créer la transaction (`statut = ENREGISTREE`, `version = 1`)

7. Créer transaction_history (action = CREATION)

8. Retourner transaction + client

L'agent ne sait jamais si le client a été créé ou retrouvé — l'API masque cette différence.

Réponse 201

```
{
  "message": "Transaction enregistrée avec succès.",
  "data": {
    "transaction": {
      "id": "uuid",
      "type_operation": "retrait",
      "montant": "70000.00",
      "reference": "OM-260912-0034",
      "solde_apres_operation": "350000.00",
      "note": null,
      "statut": "ENREGISTREE",
      "version": 1,
      "created_at": "2026-09-12T19:30:00Z"
    },
    "client": { "id": "uuid", "telephone": "70000000", "nom": "KABORE", "prenoms": "Paul" },
    "client_existant": true
  }
}
```

7.2 GET /transactions

Paramètres : telephone, search (nom/prénoms), reference, date_debut, date_fin, reseau_id, type_operation, statut, page, per_page. Tri par défaut created_at DESC.

7.3 GET /transactions/{transaction}

Retourne la transaction complète, le client, le réseau, et l'agent l'ayant enregistrée.

7.4 PATCH /transactions/{transaction}

```
{
  "montant": "75000",
  "solde_apres_operation": "355000",
  "note": "Client réclamait, montant réel confirmé",
  "motif": "Erreur de saisie initiale"
}
```

Champs modifiables (liste fermée) : montant, solde_apres_operation, note. **motif obligatoire** — conservé uniquement dans transaction_histories, jamais sur la transaction elle-même. **Interdits (ignorés silencieusement si présents) :** type_operation, reseau_id, client_id, user_id, statut, version, reference, created_at. Un dépôt et un retrait sont deux faits économiques opposés : les corriger revient à annuler puis recréer, pas à "modifier". **Fenêtre :** 24h après created_at, sinon 422.

Traitement : statut → MODIFIEE, version incrémentée, historique MODIFICATION avec anciennes/nouvelles données en JSONB et le motif fourni.

7.5 POST /transactions/{transaction}/cancel

```
{ "motif": "Erreur de montant" }
```

motif **obligatoire**. **Fenêtre :** 24h après created_at, sinon 422 (même règle que la modification, pour rester cohérent).

Traitement : statut → ANNULEE, version incrémentée, historique ANNULATION. Aucune suppression physique — DELETE /transactions/{id} n'existe pas.

8. TRAÇABILITÉ

8.1 GET /transactions/{transaction}/history

Authentification requise. Retourne toutes les entrées liées à la transaction, triées chronologiquement, sans pagination.

```
{
  "data": [
    {
      "id": "uuid",
      "action": "CREATION",
      "user": { "id": "uuid", "name": "Jean KABORE" },
      "anciennes_donnees": null,
      "nouvelles_donnees": { "montant": "70000.00",
"type_operation": "retrait" },
      "motif": null,
      "created_at": "2026-09-12T19:30:00Z"
    },
    {
      "id": "uuid",
      "action": "MODIFICATION",
      "user": { "id": "uuid", "name": "Jean KABORE" },
      "anciennes_donnees": { "montant": "70000.00" },
      "nouvelles_donnees": { "montant": "75000.00" },
      "motif": "Erreur de saisie initiale",
      "created_at": "2026-09-12T19:40:00Z"
    }
  ]
}
```

Aucun endpoint de création ou de modification directe de l'historique n'est exposé au mobile.

9. Récapitulatif des endpoints V1

Domaine	Méthode	Endpoint	Auth requise
Auth	POST	/auth/register	Non
Auth	POST	/auth/login	Non
Auth	POST	/auth/logout	Oui
Auth	GET	/auth/me	Oui
Auth	PATCH	/auth/me	Oui
Auth	POST	/auth/change-password	Oui
Auth	POST	/auth/forgot-password	Non
Auth	POST	/auth/reset-password	Non
Dashboard	GET	/dashboard	Oui
Réseaux	GET	/reseaux	Oui
Réseaux	POST	/reseaux	Oui
Réseaux	PATCH	/reseaux/{reseau}	Oui
Réseaux	DELETE	/reseaux/{reseau} (soft delete)	Oui
Clients	GET	/clients	Oui
Clients	GET	/clients/lookup	Oui
Clients	GET	/clients/{client}	Oui
Clients	GET	/clients/{client}/transactions	Oui
Clients	PATCH	/clients/{client}	Oui
Transactions	POST	/transactions	Oui
Transactions	GET	/transactions	Oui
Transactions	GET	/transactions/{transaction}	Oui
Transactions	PATCH	/transactions/{transaction}	Oui

Transactions	POST	/transactions/{transaction}/cancel	Oui
Historique	GET	/transactions/{transaction}/history	Oui

Total : 24 endpoints.

10. Endpoints volontairement absents

POST /clients → créé uniquement via une transaction
DELETE /clients/{id} → aucune suppression de client
DELETE /transactions/{id} → toujours un statut ANNULEE, jamais de suppression
POST /transaction-histories → jamais créé manuellement, uniquement par le serveur
PATCH /transaction-histories/{id} → historique immuable

Ils ne correspondent à aucun parcours utilisateur validé ou introduiraient une responsabilité inutile côté mobile.

11. Préparation à l'offline-first

Le contrat reste compatible avec une future évolution offline-first grâce à :

- des UUID générables localement par le mobile, avant tout contact avec le serveur ;
- les colonnes `version`, `sync_status`, `synced_at` déjà présentes sur `transactions`, bien qu'inutilisées en V1 ;
- l'absence de suppression physique des transactions ;
- une logique métier entièrement centralisée côté API (recherche/création client, génération de référence) ;
- l'identification du client indépendante de tout identifiant généré côté serveur (le téléphone suffit).

La V1 n'implémente aucun mécanisme de synchronisation offline — ces éléments ne font que ne pas fermer la porte à cette évolution.

12. État final

- ☒ Stratégie d'identifiants tranchée : UUID sur toutes les tables
- ☒ Module Authentification finalisé (connexion par téléphone, récupération par email, 8 endpoints)
- ☒ Module Tableau de bord finalisé (1 endpoint, aucun calcul de trésorerie)
- ☒ Module Réseaux finalisé (CRU + soft delete, pas de suppression physique)
- ☒ Module Clients finalisé (pas de création manuelle, téléphone immuable, nom/prénoms facultatifs)
- ☒ Module Transactions finalisé (référence auto-générée, champs modifiables restreints, fenêtre de 24h, motif obligatoire)
- ☒ Module Traçabilité finalisé (lecture seule, motif conservé pour chaque modification/annulation)
- ☒ Endpoints volontairement absents documentés et justifiés

Ce contrat, avec les fiches de modélisation des données et des fonctionnalités, constitue la référence complète et figée pour le développement de la V1.